

Dokumentacja Architektury Systemu

IO_RNG

System Testowania i Porównywania Generatorów Liczb Pseudolosowych

Projekt IO

14 stycznia 2026

Spis treści

1	Wprowadzenie	3
1.1	Cel dokumentu	3
1.2	Technologie	3
2	Architektura Systemu	3
2.1	Przegląd architektury	3
2.2	Schemat struktury systemu	4
2.3	Diagram klas - Core Layer	5
3	Wzorce Projektowe	6
3.1	Clean Architecture (Hexagonal Architecture)	6
3.2	Repository Pattern	6
3.3	Adapter Pattern	8
3.4	Strategy Pattern	8
3.5	Use Case Pattern	10
3.6	Dependency Injection	10
3.7	Factory Pattern (Implicit)	12
4	Zasady Programowania Obiektowego	12
4.1	SOLID Principles	12
4.1.1	Single Responsibility Principle (SRP)	12
4.1.2	Open/Closed Principle (OCP)	12
4.1.3	Liskov Substitution Principle (LSP)	13
4.1.4	Interface Segregation Principle (ISP)	13
4.1.5	Dependency Inversion Principle (DIP)	13
4.2	Enkapsulacja	14
4.3	Polimorfizm	14
4.4	Abstrakcja	14
5	Struktury Danych	15
5.1	Encje Domenowe (Dataclasses)	15
5.2	Enums (Typy wyliczeniowe)	15
5.3	Słowniki (Dict) jako Parametry	16

5.4	Listy typowane	16
5.5	Tuple dla zwracania wielu wartości	16
5.6	Optional dla wartości nullable	16
5.7	Struktury danych w bazie	17
5.7.1	Model RNG (Tabela rngs)	17
5.7.2	Model TestResult (Tabela test_results)	17
6	Diagram Przepływu Danych	19
6.1	Proces uruchomienia testu RNG	19
6.2	Proces generowania liczb - Python Runner	20
7	Szczegóły Implementacji	20
7.1	Kompresja danych - Base64	20
7.2	Testy Statystyczne	21
7.3	Obsługa różnych języków programowania	21
7.3.1	Python Runner	21
7.3.2	Executable Runner	22
8	Architektura Frontendowa	23
8.1	Struktura komponentów React	23
8.2	Wzorce frontendowe	23
8.2.1	Context API dla globalnego stanu	23
8.2.2	Custom Hooks	24
9	Komunikacja Backend-Frontend	24
9.1	REST API Endpoints	24
9.2	Przykładowe zapytanie i odpowiedź	24
10	Podsumowanie	25
10.1	Zastosowane wzorce architektoniczne	25
10.2	Zasady OOP	25
10.3	Struktury danych	26
10.4	Zalety architektury	26

1 Wprowadzenie

IO_RNG to aplikacja webowa służąca do testowania i porównywania różnych generatorów liczb pseudolosowych (PRNG). System umożliwia wykonywanie testów statystycznych na implementacjach algorytmów RNG w różnych językach programowania (Python, C++, Rust, Java) oraz analizę ich jakości i wydajności.

1.1 Cel dokumentu

Dokument przedstawia architekturę systemu, zastosowane wzorce projektowe, zasady programowania obiektowego oraz wykorzystane struktury danych.

1.2 Technologie

- **Backend:** Django REST Framework, Python 3.x
- **Frontend:** React, TypeScript, Vite
- **Baza danych:** SQLite
- **Generatory:** Python, C++, Rust, C#

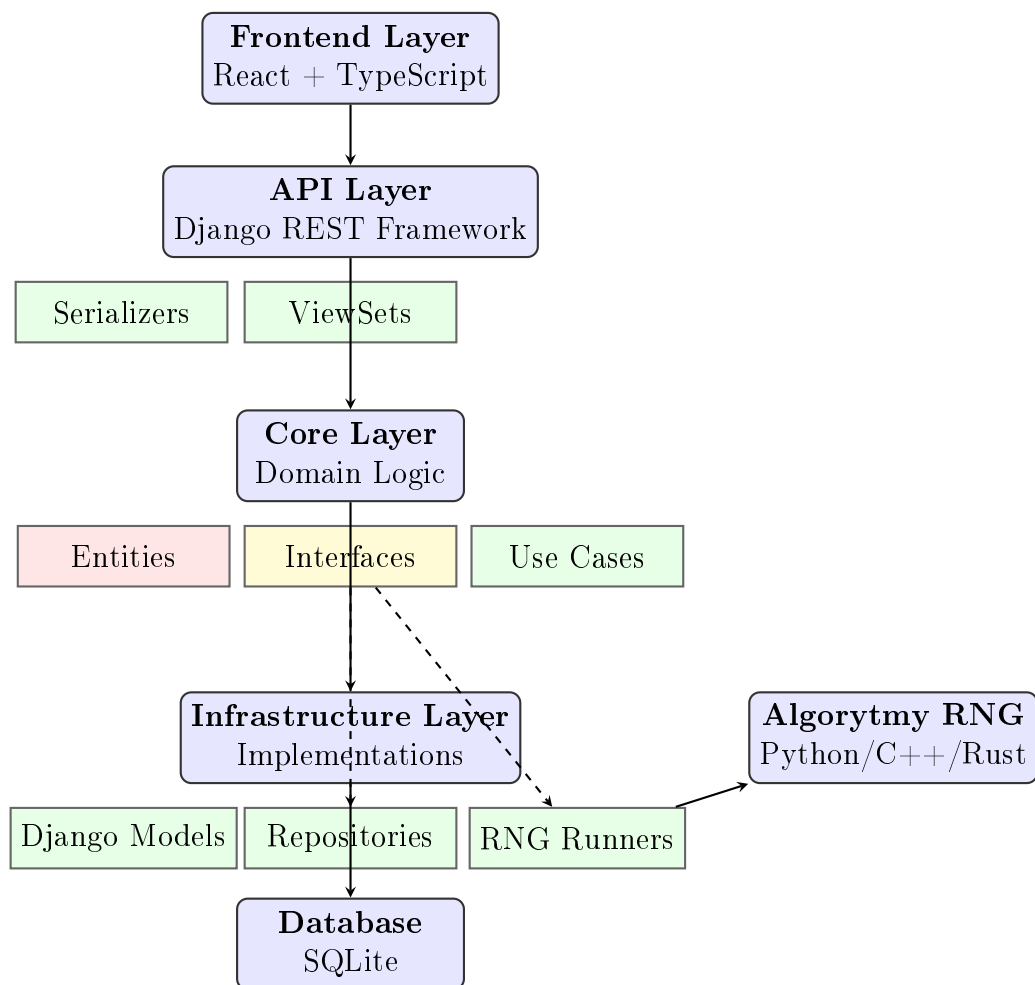
2 Architektura Systemu

2.1 Przegląd architektury

System IO_RNG implementuje **Clean Architecture** (Czystą Architekturę) z wyraźnym podziałem na warstwy:

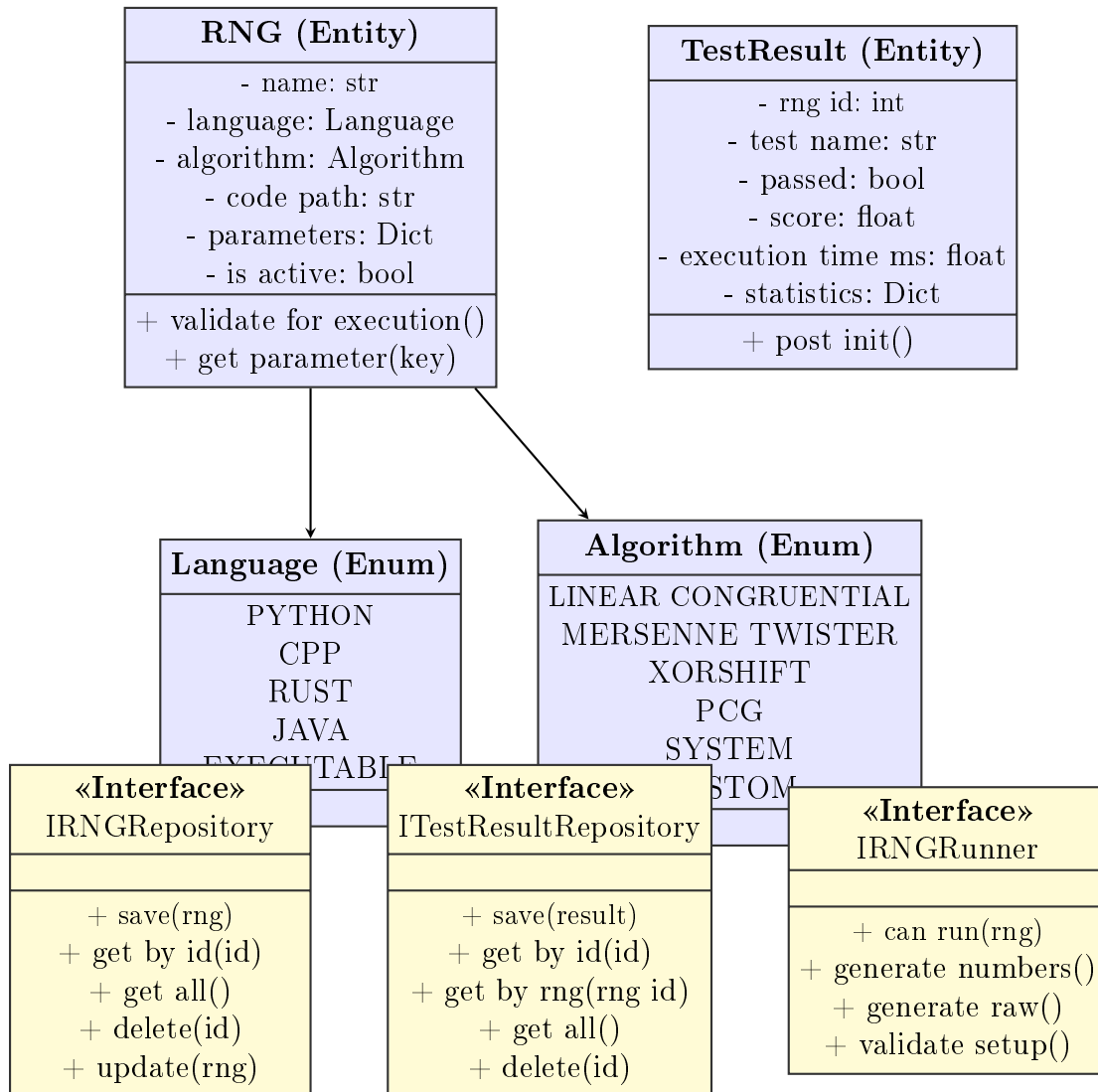
1. **Warstwa domeny (Core)** - encje i logika biznesowa
2. **Warstwa infrastruktury (Infrastructure)** - implementacje techniczne
3. **Warstwa API** - interfejs REST
4. **Warstwa prezentacji (Frontend)** - interfejs użytkownika

2.2 Schemat struktury systemu



Rysunek 1: Architektura warstwowa systemu IO_RNG

2.3 Diagram klas - Core Layer



Rysunek 2: Diagram klas warstwy Core

3 Wzorce Projektowe

System IO_RNG wykorzystuje szereg wzorców projektowych zgodnych z zasadami SOLID i Clean Architecture.

3.1 Clean Architecture (Hexagonal Architecture)

Opis: Architektura heksagonalna (Porty i Adaptery) oddziela logikę biznesową od szczegółów technicznych.

Implementacja w projekcie:

Struktura Clean Architecture

- **Warstwa Core (Domain):**
 - `entities/` - encje domenowe (RNG, TestResult)
 - `interfaces/` - porty (IRNGRepository, IRNGRunner)
 - `use_cases/` - logika biznesowa
- **Warstwa Infrastructure (Adapters):**
 - `repositories/` - implementacje repozytoriów
 - `runners/` - implementacje runnerów dla różnych języków
 - `models.py` - modele Django ORM
- **Warstwa API:**
 - `views.py` - ViewSety Django REST Framework
 - `serializers.py` - serializatory danych

Zalety:

- Niezależność logiki biznesowej od frameworka
- Łatwość testowania (możliwość mockowania interfejsów)
- Możliwość łatwej zamiany implementacji (np. zmiana bazy danych)

3.2 Repository Pattern

Cel: Abstrakcja dostępu do danych, oddzielenie logiki biznesowej od mechanizmów persistencji.

Implementacja:

```
1 class IRNGRepository(ABC):
2     @abstractmethod
3     def save(self, rng: RNG) -> RNG:
4         pass
5
6     @abstractmethod
7     def get_by_id(self, rng_id: int) -> Optional[RNG]:
8         pass
```

```

9
10 @abstractmethod
11 def get_all(self) -> List[RNG]:
12     pass

```

Listing 1: Interface repozytorium

```

1 class DjangoRNGRepository(IRNGRepository):
2     def get_by_id(self, rng_id: int) -> Optional[RNG]:
3         try:
4             model = RNGModel.objects.get(id=rng_id)
5             return self._to_entity(model)
6         except RNGModel.DoesNotExist:
7             return None
8
9     def _to_entity(self, model: RNGModel) -> RNG:
10        return RNG(
11            id=model.id,
12            name=model.name,
13            language=Language(model.language),
14            algorithm=Algorithm(model.algorithm),
15            # ... inne pola
16        )

```

Listing 2: Implementacja dla Django ORM

Zalety:

- Enkapsulacja logiki dostępu do danych
- Możliwość łatwej zamiany źródła danych (SQLite → PostgreSQL)
- Ułatwienie testowania (mock repositories)

3.3 Adapter Pattern

Cel: Umożliwienie współpracy niezgodnych interfejsów.

Implementacja - Universal RNG Adapter:

System używa wzorca Adapter do automatycznego wykrywania i adaptacji różnych implementacji generatorów:

```
1 class UniversalRNGAdapter:
2     def __init__(self, module, parameters: Optional[Dict] = None):
3         self.module = module
4         self.parameters = parameters or {}
5         self.generator_func = self._detect_generator_function()
6         self.generator_type = self._detect_generator_type()
7
8     def _detect_generator_function(self):
9         # Priorytet 1: generate()
10        if hasattr(self.module, 'generate'):
11            return self.module.generate
12
13        # Priorytet 2: *_bit_stream()
14        for name in dir(self.module):
15            if name.endswith('_bit_stream'):
16                return getattr(self.module, name)
17
18        raise RuntimeError("No generator function found")
19
20    def _detect_generator_type(self) -> DataType:
21        # Wykrywa typ danych: BITS, INTEGERS, FLOATS
22        # ...
```

Listing 3: Universal RNG Adapter

Zalety:

- Automatyczne dopasowanie do różnych sygnatur funkcji
- Obsługa różnych typów danych wyjściowych (bity, integer, floaty)
- Brak konieczności modyfikacji istniejących generatorów

3.4 Strategy Pattern

Cel: Umożliwienie wyboru algorytmu w czasie wykonania.

Implementacja - RNG Runners:

```
1 class IRNGRunner(ABC):
2     @abstractmethod
3     def can_run(self, rng: RNG) -> bool:
4         pass
5
6     @abstractmethod
7     def generate_numbers(self, rng: RNG, count: int,
8                           seed: int = None) -> List[float]:
9         pass
```

Listing 4: Interface strategii


```

1 class PythonRNGRunner(IRNGRunner):
2     def can_run(self, rng: RNG) -> bool:
3         return rng.language == Language.PYTHON
4     # ...
5
6 class ExeRNGRunner(IRNGRunner):
7     def can_run(self, rng: RNG) -> bool:
8         return rng.language == Language.EXECUTABLE
9     # ...

```

Listing 5: Konkretne strategie

Wybór strategii w runtime:

```

1 def _find_runner(self, rng: RNG) -> IRNGRunner:
2     for runner in self.runners:
3         if runner.can_run(rng):
4             return runner
5     raise RuntimeError(f"No runner for {rng.language}")

```

3.5 Use Case Pattern

Cel: Enkapsulacja logiki biznesowej w pojedynczych, specyficznych przypadkach użycia.

Implementacja:

```
1 class RunRNGTestUseCase:
2     def __init__(self,
3                 rng_repository: IRNGRepository,
4                 result_repository: ITestResultRepository,
5                 runners: List[IRNGRunner]):
6         self.rng_repository = rng_repository
7         self.result_repository = result_repository
8         self.runners = runners
9
10    def execute(self, rng_id: int, test_name: str,
11               samples_count: int, seed: int = None) -> TestResult:
12        # 1. Pobierz RNG z repozytorium
13        rng = self.rng_repository.get_by_id(rng_id)
14
15        # 2. Znajdz odpowiedni runner
16        runner = self._find_runner(rng)
17
18        # 3. Generuj liczby losowe
19        numbers = runner.generate_numbers(rng, samples_count, seed)
20
21        # 4. Wykonaj test statystyczny
22        test_result = self._perform_statistical_test(...)
23
24        # 5. Zapisz wynik
25        return self.result_repository.save(result)
```

Listing 6: Run RNG Test Use Case

Inne Use Cases w systemie:

- CompareRNGsUseCase - porównywanie generatorów
- GetTestResultsUseCase - pobieranie wyników testów
- TestCustomBitsUseCase - testowanie własnych sekwencji bitów

3.6 Dependency Injection

Cel: Odwrócenie kontroli, zwiększenie testowalności.

Implementacja w ViewSet:

```
1 class RNGViewSet(viewsets.ViewSet):
2     def __init__(self, **kwargs):
3         super().__init__(**kwargs)
4         # Tworzenie konkretnych implementacji
5         self.rng_repository = DjangoRNGRepository()
6         self.result_repository = DjangoTestResultRepository()
7         self.runners = [
8             PythonRNGRunner(),
9             ExeRNGRunner()
10        ]
11
12    @action(detail=True, methods=['post'])
13    def run_test(self, request, pk=None):
```

```
14         # Wstrzykiwanie do Use Case
15         use_case = RunRNGTestUseCase(
16             self.rng_repository,
17             self.result_repository,
18             self.runners
19         )
20         result = use_case.execute(...)
21         return Response(...)
```

Listing 7: Wstrzykiwanie zależności

3.7 Factory Pattern (Implicit)

System używa niejawnego wzorca Factory w procesie tworzenia encji z modeli Django:

```
1 def _to_entity(self, model: RNGModel) -> RNG:
2     """Factory method - creates entity from Django model"""
3     return RNG(
4         id=model.id,
5         name=model.name,
6         language=Language(model.language),
7         algorithm=Algorithm(model.algorithm),
8         description=model.description,
9         code_path=model.code_path,
10        parameters=model.parameters or {},
11        is_active=model.is_active
12    )
```

Listing 8: Konwersja Model do Entity

4 Zasady Programowania Obiektowego

4.1 SOLID Principles

System IO_RNG konsekwentnie stosuje zasady SOLID:

4.1.1 Single Responsibility Principle (SRP)

Każda klasa ma jedną, dobrze zdefiniowaną odpowiedzialność:

Przykłady:

- RNG - reprezentuje generator (encja domeny)
- DjangoRNGRepository - zarządza persystencją RNG
- PythonRNGRunner - wykonuje generatory Python
- RunRNGTestUseCase - orkiestruje proces testowania
- UniversalRNGAdapter - adaptuje różne formaty generatorów

4.1.2 Open/Closed Principle (OCP)

Klasy są otwarte na rozszerzenia, zamknięte na modyfikacje:

```
1 # Adding support for new language without modifying existing code
2 class RustRNGRunner(IRNGRunner):
3     def can_run(self, rng: RNG) -> bool:
4         return rng.language == Language.RUST
5
6     def generate_numbers(self, rng: RNG, ...) -> List[float]:
7         # Rust implementation
8         pass
9
10 # In ViewSet - just add to list
```

```

11 self.runners = [
12     PythonRNGRunner(),
13     ExeRNGRunner(),
14     RustRNGRunner() # New runner
15 ]

```

Listing 9: Przykład OCP - dodanie nowego runnera

4.1.3 Liskov Substitution Principle (LSP)

Wszystkie implementacje interfejsów są podstawialne:

```

1 # Every IRNGRunner implementation is substitutable
2 def test_with_any_runner(runner: IRNGRunner, rng: RNG):
3     if runner.can_run(rng):
4         numbers = runner.generate_numbers(rng, 1000)
5         # Works with PythonRNGRunner, ExeRNGRunner, etc.

```

4.1.4 Interface Segregation Principle (ISP)

Interfejsy są minimalistyczne i skoncentrowane:

- IRNGRepository - tylko operacje na RNG
- ITestResultRepository - tylko operacje na wynikach
- IRNGRunner - tylko generowanie liczb

4.1.5 Dependency Inversion Principle (DIP)

Moduły wysokopoziomowe zależą od abstrakcji, nie od konkretnych implementacji:

```

1 class RunRNGTestUseCase:
2     # Depends on abstractions (interfaces)
3     def __init__(self,
4         rng_repository: IRNGRepository, # Abstraction
5         result_repository: ITestResultRepository, #
6         Abstraction
7         runners: List[IRNGRunner]): # Abstraction
8         # ...

```

4.2 Enkapsulacja

Enkapsulacja danych i logiki w encjach:

```
1 @dataclass
2 class RNG:
3     # Public attributes (dataclass)
4     name: str
5     language: Language
6     algorithm: Algorithm
7     # ...
8
9     def __post_init__(self):
10         """Private validation - called automatically"""
11         if not self.name:
12             raise ValueError("RNG name cannot be empty")
13         if not self.code_path:
14             raise ValueError("Code path cannot be empty")
15
16     def validate_for_execution(self) -> bool:
17         """Public business method"""
18         return self.is_active and bool(self.code_path)
19
20     def get_parameter(self, key: str, default: Any = None) -> Any:
21         """Safe access to parameters"""
22         return self.parameters.get(key, default)
```

Listing 10: Enkapsulacja w RNG Entity

4.3 Polimorfizm

Polimorfizm w runnerach:

```
1 # Common interface
2 numbers = runner.generate_numbers(rng, count, seed)
3
4 # PythonRNGRunner - loads Python module and executes
5 # ExeRNGRunner - runs .exe file and parses stdout
6 # CppRNGRunner - compiles and runs C++ code
7 # All return List[float]
```

Listing 11: Różne implementacje tego samego interfejsu

4.4 Abstrakcja

System używa abstrakcji na wielu poziomach:

- **Encje** - abstrakcja koncepcji biznesowych (RNG, TestResult)
- **Interfejsy** - abstrakcja operacji (IRNGRunner, IRNGRepository)
- **Enumy** - abstrakcja wartości (Language, Algorithm, DataType)

5 Struktury Danych

5.1 Encje Domenowe (Dataclasses)

System używa Python dataclasses jako immutable value objects:

```
1 @dataclass
2 class RNG:
3     name: str
4     language: Language
5     algorithm: Algorithm
6     description: str
7     code_path: str
8     parameters: Optional[Dict[str, Any]] = None
9     id: Optional[int] = None
10    is_active: bool = True
```

Listing 12: Struktura encji RNG

Zalety dataclasses:

- Automatyczna generacja `__init__`, `__repr__`, `__eq__`
- Type hints dla wszystkich pól
- Niezmienność (immutability) z `frozen=True`
- Walidacja w `__post_init__`

5.2 Enumy (Typy wyliczeniowe)

System definiuje silnie typowane enumy dla wartości biznesowych:

```
1 class Language(Enum):
2     PYTHON = "python"
3     JAVA = "java"
4     CPP = "cpp"
5     RUST = "rust"
6     EXECUTABLE = "executable"
7
8 class Algorithm(Enum):
9     LINEAR_CONGRUENTIAL = "linear_congruential"
10    MERSENNE_TWISTER = "mersenne_twister"
11    XORSHIFT = "xorshift"
12    PCG = "pcg"
13    SYSTEM = "system"
14
15 class DataType(Enum):
16    BITS = "bits"
17    INTEGERS = "integers"
18    FLOATS = "floats"
```

Listing 13: Enumy domenowe

Zalety enumów:

- Bezpieczeństwo typów
- Niemożliwość użycia nieprawidłowych wartości

- Autocomplete w IDE
- Czytelność kodu

5.3 Słowniki (Dict) jako Parametry

System używa słowników JSON do przechowywania dynamicznych parametrów:

```

1 # Example LCG parameters
2 parameters = {
3     "a": 1103515245,      # multiplier
4     "c": 12345,          # increment
5     "m": 2**31,          # modulus
6     "bits_per_value": 31 # number of bits
7 }
8
9 # Safe access via method
10 def get_parameter(self, key: str, default: Any = None) -> Any:
11     return self.parameters.get(key, default) if self.parameters else
    default

```

Listing 14: Parametry generatorow

5.4 Listy typowane

System używa list z type hints dla bezpieczeństwa typów:

```

1 # List of bits
2 bits: List[int] = [0, 1, 0, 1, 1, 0, ...]
3
4 # List of floats
5 numbers: List[float] = [0.234, 0.891, 0.456, ...]
6
7 # List of entities
8 rngs: List[RNG] = repository.get_all()
9
10 # List of runners
11 runners: List[IRNGRunner] = [PythonRNGRunner(), ExeRNGRunner()]

```

5.5 Tuple dla zwracania wielu wartości

```

1 def generate_raw(self, rng: RNG, count: int,
2                 seed: int = None) -> Tuple[List[Any], DataType]:
3     """Returns (data, data_type)"""
4     # ...
5     return (data, DataType.BITS)

```

Listing 15: Tuple z type hints

5.6 Optional dla wartości nullable

```

1 def get_by_id(self, rng_id: int) -> Optional[RNG]:
2     """Can return RNG or None"""
3     try:
4         model = RNGModel.objects.get(id=rng_id)

```



```

5         return self._to_entity(model)
6     except RNGModel.DoesNotExist:
7         return None

```

5.7 Struktury danych w bazie

5.7.1 Model RNG (Tabela rngs)

```

1 class RNGModel(models.Model):
2     name = models.CharField(max_length=200)
3     language = models.CharField(max_length=50)
4     algorithm = models.CharField(max_length=100)
5     description = models.TextField()
6     code_path = models.CharField(max_length=500)
7     parameters = models.JSONField(null=True, blank=True)
8     is_active = models.BooleanField(default=True)
9     created_at = models.DateTimeField(auto_now_add=True)
10
11     class Meta:
12         db_table = 'rngs'
13         ordering = ['-created_at']

```

Listing 16: Django Model

Indeksy:

- Primary key na id
- Ordering index na created_at

5.7.2 Model TestResult (Tabela test_results)

```

1 class TestResultModel(models.Model):
2     rng = models.ForeignKey(RNGModel, on_delete=models.CASCADE,
3                             related_name='test_results')
4     test_name = models.CharField(max_length=100)
5     passed = models.BooleanField()
6     score = models.FloatField()
7     execution_time_ms = models.FloatField()
8     samples_count = models.IntegerField()
9     statistics = models.JSONField()
10    error_message = models.TextField(null=True, blank=True)
11    created_at = models.DateTimeField(auto_now_add=True)
12
13    class Meta:
14        db_table = 'test_results'
15        ordering = ['-created_at']
16        indexes = [
17            models.Index(fields=['rng', '-created_at']),
18            models.Index(fields=['test_name', '-created_at']),
19        ]

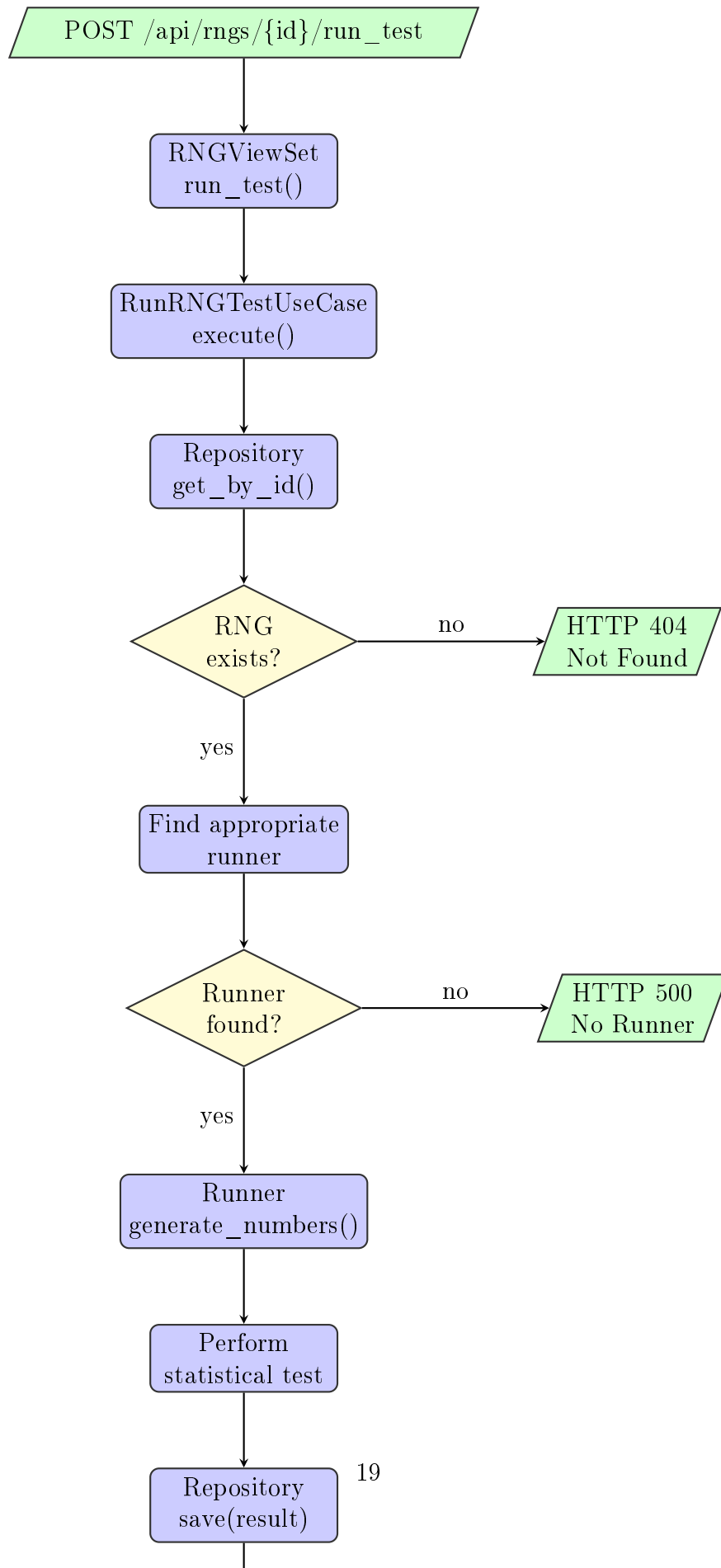
```

Indeksy optymalizacyjne:

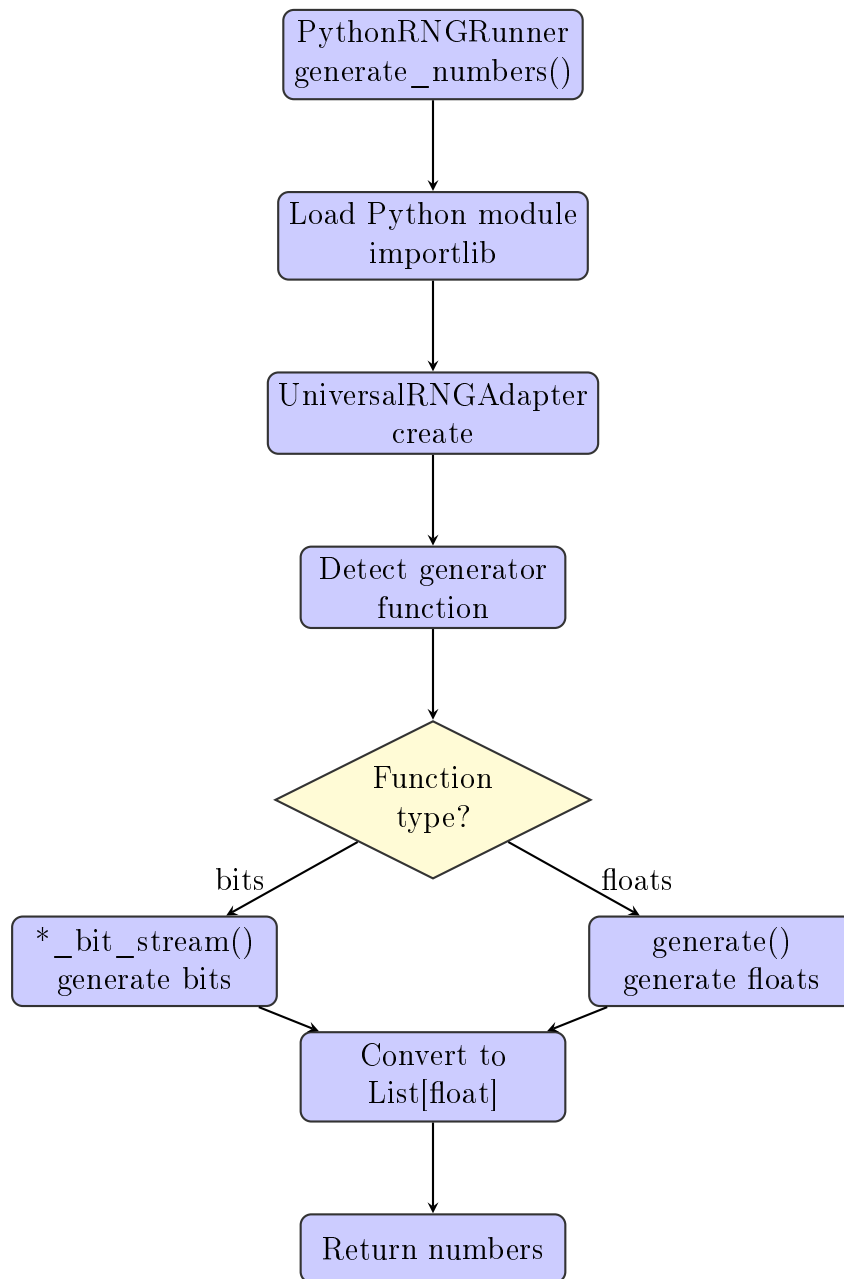
- Composite index: (rng_id, created_at) - szybkie pobieranie wyników dla RNG
- Composite index: (test_name, created_at) - filtrowanie po typie testu

6 Diagram Przepływu Danych

6.1 Proces uruchomienia testu RNG



6.2 Proces generowania liczb - Python Runner



Rysunek 4: Proces generowania liczb w Python Runner

7 Szczegóły Implementacji

7.1 Kompresja danych - Base64

System używa kompresji bitów do Base64 dla oszczędności transferu danych:

```
1 def compress_bits_to_base64(bits: List[int]) -> str:
2     """
3     Packs bits list [0,1,0,1,...] to base64.
4     Savings: ~8x (3 bytes/bit to 0.375 bytes/bit)
5     """
```

```

6     byte_array = bytearray()
7
8     for i in range(0, len(bits), 8):
9         byte_val = 0
10        for j in range(8):
11            if i + j < len(bits):
12                # MSB first
13                byte_val |= (bits[i + j] << (7 - j))
14            byte_array.append(byte_val)
15
16    return base64.b64encode(byte_array).decode('ascii')

```

Listing 17: Kompresja bitów

Przykład:

- Bez kompresji: [1,0,1,0,1,0,1,0] → 24 bajty JSON
- Z kompresją: "qg==" → 4 bajty
- Oszczędność: 83%

7.2 Testy Statystyczne

System implementuje szereg testów statystycznych:

1. **Chi-Square Test** - test zgodności rozkładu
2. **Kolmogorov-Smirnov Test** - test równomierności
3. **Runs Test** - test niezależności sekwencji
4. **Spectral Test (FFT)** - test częstotliwościowy
5. **Poker Test** - test sekwencji 5-bitowych
6. **Monobit Test** - test proporcji bitów
7. **Serial Test** - test par bitów
8. **Autocorrelation Test** - test autokorelacji

7.3 Obsługa różnych języków programowania

7.3.1 Python Runner

- Dynamiczne ładowanie modułów: `importlib.util`
- Automatyczna detekcja funkcji generatora
- Wsparcie dla parametryzowanych generatorów (LCG, AWCG)

7.3.2 Executable Runner

- Uruchamianie prekompilowanych plików .exe
- Komunikacja przez stdout/stdin
- Parsowanie wyjścia tekstowego
- Wsparcie dla Rust, C++, C#

8 Architektura Frontendowa

8.1 Struktura komponentów React

Frontend używa architektury komponentowej z podziałem funkcjonalnym:

Struktura komponentów

- **Layout:**
 - `NavigationSidebar` - nawigacja boczna
 - `Footer` - stopka
 - `Header` - nagłówek
- **Pages (Routes):**
 - `Dashboard` - pulpit główny
 - `Generator` - generator liczb
 - `Tests` - testy statystyczne
 - `Results` - wyniki testów
 - `Wiki` - dokumentacja algorytmów
 - `Settings` - ustawienia
- **UI Components (shadcn/ui):**
 - `Button`, `Card`, `Dialog`, `Select`
 - `Table`, `Tabs`, `Progress`
 - `Chart` (Recharts)
- **Context Providers:**
 - `ThemeProvider` - motyw (dark/light)
 - `LanguageProvider` - internacjonalizacja
 - `TestResultsProvider` - stan wyników testów

8.2 Wzorce frontendowe

8.2.1 Context API dla globalnego stanu

```
1 export const TestResultsProvider = ({ children }) => {
2   const [results, setResults] = useState<TestResult[]>([]);
3   const [loading, setLoading] = useState(false);
4
5   const fetchResults = async () => {
6     setLoading(true);
7     const data = await api.getTestResults();
8     setResults(data);
9     setLoading(false);
10  };
```

```

11
12 return (
13   <TestResultsContext.Provider value={{
14     results, loading, fetchResults
15   }}>
16     {children}
17   </TestResultsContext.Provider>
18 );
19 };

```

Listing 18: Test Results Context

8.2.2 Custom Hooks

System używa custom hooks dla logiki wielokrotnego użytku:

- `useRNGList()` - pobieranie listy generatorów
- `useTestExecution()` - uruchamianie testów
- `useLanguage()` - zarządzanie językiem interfejsu
- `useTheme()` - zarządzanie motywem

9 Komunikacja Backend-Frontend

9.1 REST API Endpoints

Metoda	Endpoint	Opis
GET	/api/rngs	Lista wszystkich RNG
POST	/api/rngs	Utworzenie nowego RNG
GET	/api/rngs/{id}	Szczegóły RNG
PUT	/api/rngs/{id}	Aktualizacja RNG
DELETE	/api/rngs/{id}	Usunięcie RNG
POST	/api/rngs/{id}/run_test	Uruchomienie testu
POST	/api/rngs/{id}/generate	Generowanie liczb bez testu
GET	/api/rngs/{id}/test_results	Wyniki testów dla RNG
GET	/api/test-results	Lista wszystkich wyników
GET	/api/test-results/{id}	Szczegóły wyniku
DELETE	/api/test-results/{id}	Usunięcie wyniku

Tabela 1: REST API Endpoints

9.2 Przykładowe zapytanie i odpowiedź

Request:

```

1 POST /api/rngs/1/run_test
2 Content-Type: application/json
3
4 {
5   "test_name": "chi_square",

```



```

6  "samples_count": 100000,
7  "seed": 42,
8  "parameters": {
9    "a": 1103515245,
10   "c": 12345,
11   "m": 2147483648
12 }
13 }

```

Response:

```

1  {
2    "id": 123,
3    "rng_id": 1,
4    "test_name": "chi_square",
5    "passed": true,
6    "score": 0.87,
7    "execution_time_ms": 234.56,
8    "samples_count": 100000,
9    "statistics": {
10     "chi_square_statistic": 12.34,
11     "p_value": 0.87,
12     "degrees_of_freedom": 9
13   },
14   "created_at": "2026-01-14T10:30:00Z"
15 }

```

Listing 19: JSON Response

10 Podsumowanie

10.1 Zastosowane wzorce architektoniczne

1. **Clean Architecture** - podział na warstwy: Core, Infrastructure, API
2. **Repository Pattern** - abstrakcja dostępu do danych
3. **Adapter Pattern** - Universal RNG Adapter
4. **Strategy Pattern** - wymienne runnery dla języków
5. **Use Case Pattern** - enkapsulacja logiki biznesowej
6. **Dependency Injection** - wstrzykiwanie zależności
7. **MVC/MVP** - separacja widoku i logiki (Frontend)

10.2 Zasady OOP

1. SOLID:

- **Single Responsibility** - każda klasa ma jedną odpowiedzialność
- **Open/Closed** - otwarte na rozszerzenia, zamknięte na modyfikacje
- **Liskov Substitution** - implementacje podstawialne
- **Interface Segregation** - minimalistyczne interfejsy

- **Dependency Inversion** - zależność od abstrakcji
2. **Enkapsulacja** - dane i logika ukryte w klasach
 3. **Polimorfizm** - różne implementacje tego samego interfejsu
 4. **Abstrakcja** - ukrycie szczegółów implementacji
 5. **Dziedziczenie** - przez interfejsy (ABC)

10.3 Struktury danych

1. **Dataclasses** - encje domenowe (RNG, TestResult)
2. **Enumy** - typy wyliczeniowe (Language, Algorithm, DataType)
3. **Dict** - parametry dynamiczne, JSON
4. **List[T]** - kolekcje typowane
5. **Tuple** - zwracanie wielu wartości
6. **Optional[T]** - wartości nullable
7. **Django Models** - ORM dla persystencji
8. **JSONField** - elastyczne przechowywanie danych

10.4 Zalety architektury

- **Testowalność** - łatwe mockowanie interfejsów
- **Rozszerzalność** - łatwe dodawanie nowych runnerów, testów, algorytmów
- **Niezależność od frameworka** - logika biznesowa w Core
- **Separacja warstw** - clear separation of concerns
- **Type Safety** - type hints w Python, TypeScript w frontend
- **Maintainability** - czytelna struktura, SOLID